

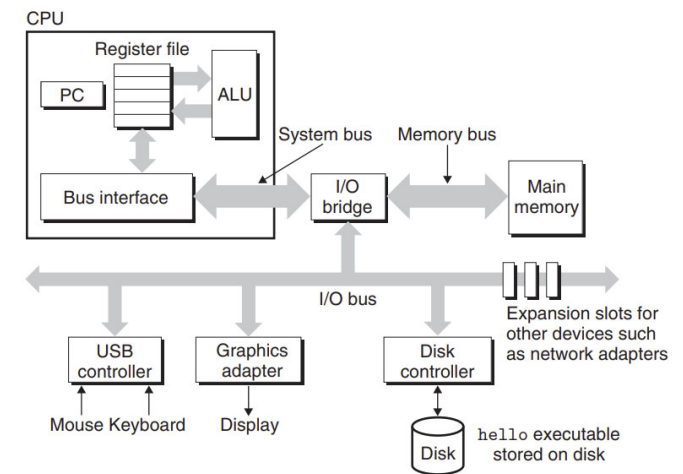
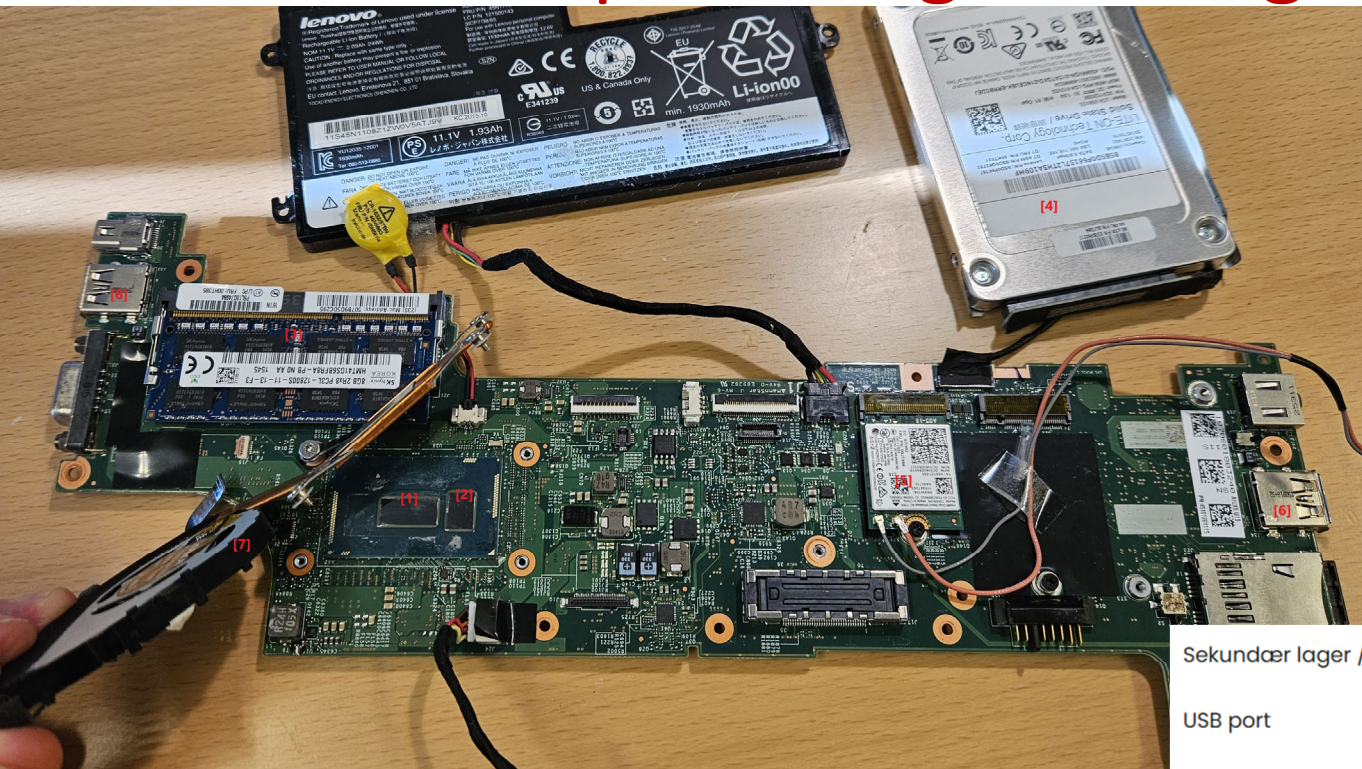
Computer Arkitektur

Bits, Bytes, og Integers

Forelæsning 1
Brian Nielsen

*Credits to
Randy Bryant & Dave O'Hallaron (CMU)*

Quizz: Computer Organisering - 1



Sekundær lager / Disk

USB port

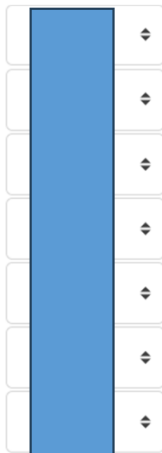
CPU / processor (specifikt Intel i5)

I/O- og System-Bridge alias PCH (Platform Controller HUB)

Primær Hukommelsen / main-memory / arbejdshukommelsen / RAM

Blæser og køle-element

Wifi Adapter



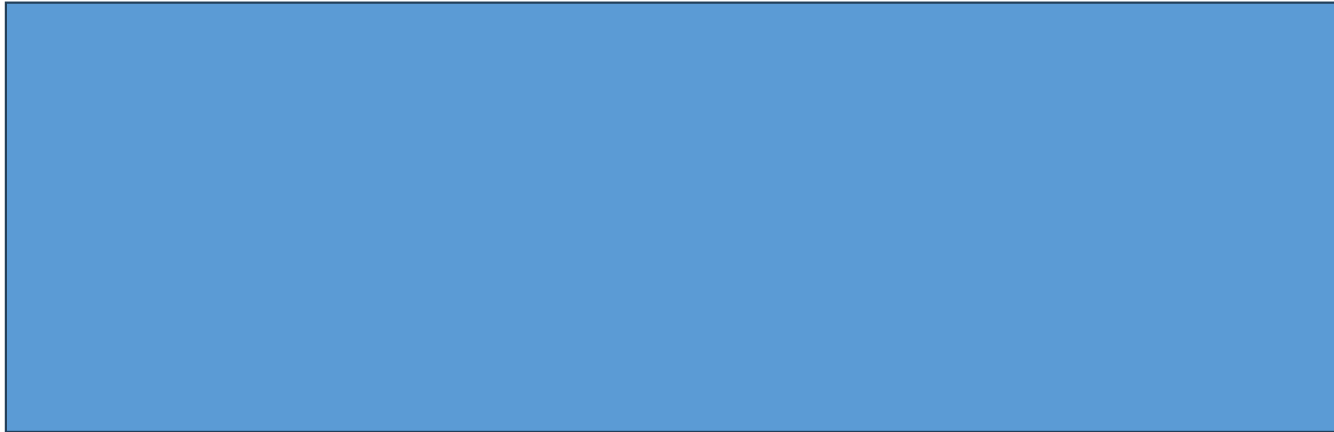
Quizz: Computer Organisering - 2

Question 2

Not yet answered

Marked out of 3.00

Hvad er formålet med kølelementet og hvorfor er det nødvendigt?



Quizz: Computer Organisering - 3

Question 3

Not yet answered

Marked out of 3.00

Hvor på printpladen findes mon "system-" "memory-" og "I/O-busserne"?

A large blue rectangular area, likely a placeholder for an image or diagram related to the question. It occupies the majority of the lower half of the page.

Plenum

- Quizz: Computer HW Organisering
- Hex fejl i video
- Notation for binære værdier (+udskrivning af bits)
- Xor
- Brug af boolsk algebra til reduktion af udtryk
- Quizz: Tal-repræsentation
- Quizz: Endianess

Bug i video-slide



Hexa-decimale tal

- Base 16
 - Streng af cifrene '0' til '9' og 'A' til 'F'
 - $1AF_{16} = 1 \cdot 16^2 + A \cdot 16^1 + F \cdot 16^0$
 $= 1 \cdot 16^2 + 10 \cdot 16^1 + 15 \cdot 16^0 = 431_{10}$

- C-notation: hex tal præfixes med "0x"

FA1D37B

- $FA1D37_{16}$ skrives som
 - 0xFA1D37B , eller 0xfa1d37b

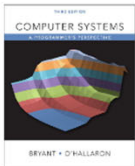
- Konvertering til binært er NEMT
 - Erstat hvert hex-tal med dets binære værdi (4 bits)
 - $2BA \rightarrow 0010\ 1011\ 1010$
- Konvertering fra binært er NEMT
 - Opdel bitstreng i grupper af 4 (højre mod venstre)
 - $1010111010 \rightarrow 10\ 1011\ 1010 \rightarrow 2BA$

Hex	Decimal	Binær
0	0	0000
1	1	0001
2	2	0010
3	3	0011
4	4	0100
5	5	0101
6	6	0110
7	7	0111
8	8	1000
9	9	1001
A	10	1010
B	11	1011
C	12	1100
D	13	1101
E	14	1110
F	15	1111

- Investér tid nu til at blive fortrolig med notationen og konvertering.
- NEM konvertering HEX-BIN

Apropos bugs ... i opgaver og løsninger

← → ↻ 🏠 ⚠ Not secure | csapp.cs.cmu.edu/3e/errata.html



Computer Systems: A Programmer's Perspective, 3/E (CS:APP3e)

Randal E. Bryant and David R. O'Hallaron, Carnegie Mellon University

Home
Web Asides
Student Site
Instructor Site
TOC and Preface
Adoptions
Errata
Papers
Curriculum
Courses

Errata for CS:APP3e and its Instructors Manual

Last updated 10/02/2020.

Despite our best efforts to create a book with zero defects, our vigilant readers have pointed out some bugs. Please report any new errata to [Randy Bryant](#) and [Dave O'Hallaron](#). Note that some of these errors have been corrected in more recent printings.

We maintain errata for the North American edition. Errata for editions in other languages can be found as follows:

- [Chinese](#), maintained by [Prof. Yili Gong](#) of [Wuhan University](#).

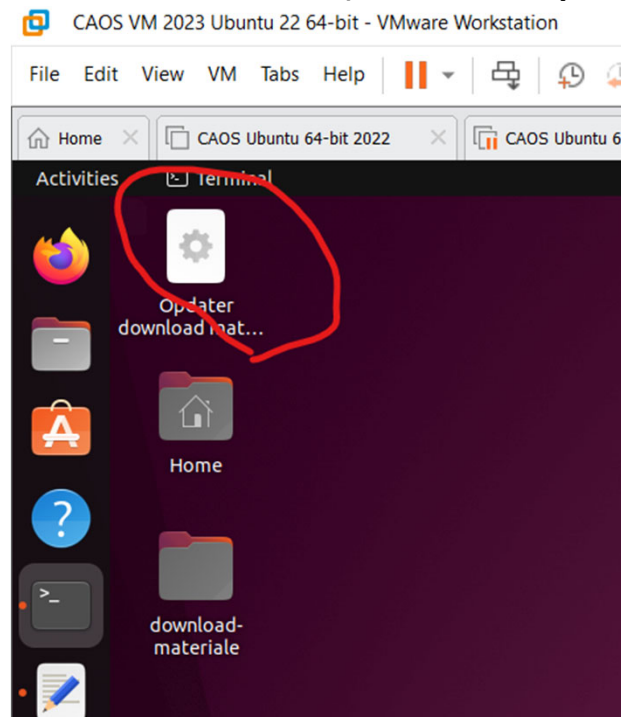
Note on the Global Edition: Unfortunately, the publisher arranged for the generation of a different set of practice and homework problems in the global edition. The person doing this didn't do a very good job, and so these problems and their solutions have many errors. We have not created an errata for this edition.

North American Edition (ISBN-10: 0-13-409206-X)

- Please send complaints to Pearson (publisher)
- Se Moodle for korrektion af (de udpegede) opgaver og løsninger.
- Hovedtekst er OK!

Praktiske Øvelser “Challenges”

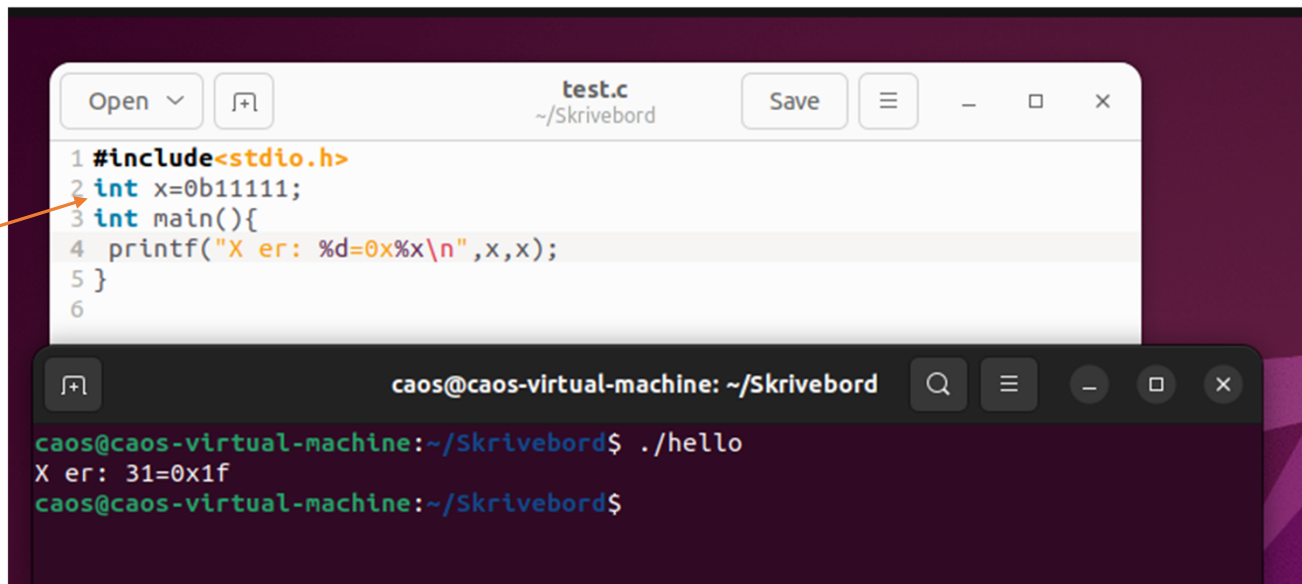
- Skal laves om alle andre opgaver
- Findes i VM (download-materiale). Kør “Opdater Script”



```
wget -m --no-parent --no-host-directories -P . --cut-dirs=2 --reject "index.html*" https://homes.cs.aau.dk/~bnielsen/CART/;
```

Notation for binære værdier

- **0x** el. **0X** for hex i C
- Nogle compilere accepterer **0b** el. **0B** for binære værdier



The image shows a code editor window titled 'test.c' with the following C code:

```
1 #include<stdio.h>
2 int x=0b111111;
3 int main(){
4     printf("X er: %d=0x%x\n",x,x);
5 }
6
```

An orange arrow points to the binary value `0b111111` on line 2. Below the code editor is a terminal window with the following output:

```
caos@caos-virtual-machine: ~/Skrivebord
caos@caos-virtual-machine:~/Skrivebord$ ./hello
X er: 31=0x1f
caos@caos-virtual-machine:~/Skrivebord$
```

Boolsk Algebra

- Udviklet af George Boole ca. år 1850
 - Algebra over Sand-Falsk værdier
 - Konvention: "True" kodes som 1, og "False" som 0

True, False
$\wedge$ AND
$\vee$ OR
$\neg$ NOT
$\oplus$ XOr

And

$A \& B = 1$ når både $A=1$ og $B=1$

&		B	
		0	1
A	0	0	0
	1	0	1

Or

$A \mid B = 1$ når $A=1$ eller $B=1$

		B	
		0	1
A	0	0	1
	1	1	1

Exclusive-Or (Xor: "enten-eller")

$A \wedge B = 1$ når netop ét af: $A=1$ eller $B=1$,

$\wedge$		B	
		0	1
A	0	0	1
	1	1	0

Not

$\sim A$ = negeres

$\sim$		
A	0	1
	1	0

$$A \wedge B = A \& \sim B \mid \sim A \& B$$

$$A \wedge A = A \& \sim A \mid \sim A \& A = 0$$

General Boolsk Algebra

Commutative:

- $A \& B = B \& A$
- $A \mid B = B \mid A$

Associative:

- $(A \& B) \& C = A \& (B \& C)$
- $(A \mid B) \mid C = A \mid (B \mid C)$

Distributive:

- $(A \& (B \mid C)) = (A \& B) \mid (A \& C)$
- $(A \mid (B \& C)) = (A \mid B) \& (A \mid C)$

Identity:

- $A \& 1 = A$
- $A \mid 0 = A$

Complement:

- $(A \& (\sim A)) = 0$
- $A \mid (\sim A) = 1$

DeMorgan's Law:

- $\sim (A \& B) = (\sim A) \mid (\sim B)$
- $\sim (A \mid B) = (\sim A) \& (\sim B)$

Eksempel på reduktion af udtryk

$$\begin{aligned} & \sim (A \& \sim B) \mid \sim (\sim A \mid B) \& C \\ \Downarrow & \\ & \sim A \mid \sim \sim B \mid \sim (\sim A \mid B) \& C \\ \Downarrow & \\ & \sim A \mid \sim \sim B \mid (\sim \sim A \& \sim B) \& C \\ \Downarrow & \\ & \sim A \mid B \mid (A \& \sim B) \& C \\ \Downarrow & \\ & \sim A \mid B \mid A \& \sim B \& C \end{aligned}$$

//DeMorgan

//DeMorgan

// $\sim \sim x = x$

//assoc.

Print as bit-strings

```
#include<stdio.h>
#include<limits.h>

#define BYTE_TO_BINARY_PATTERN "%c%c%c%c%c%c%c%c"
#define BYTE_TO_BINARY(byte)  \
    (byte & 0x80 ? '1' : '0'), \
    (byte & 0x40 ? '1' : '0'), \
    (byte & 0x20 ? '1' : '0'), \
    (byte & 0x10 ? '1' : '0'), \
    (byte & 0x08 ? '1' : '0'), \
    (byte & 0x04 ? '1' : '0'), \
    (byte & 0x02 ? '1' : '0'), \
    (byte & 0x01 ? '1' : '0')

//cite: https://stackoverflow.com/questions/111928/is-there-a-
printf-converter-to-print-in-binary-format
int shift(){
    char x=-128; //chars are signed!
    unsigned char y =128;

    printf("x=%d\t\tty=%u\n",x,y);
    printf("x: "BYTE_TO_BINARY_PATTERN"\ty:"BYTE_TO_BINARY_PATTERN"\n",
    BYTE_TO_BINARY(x) ,BYTE_TO_BINARY(y));
    x=x>>4;
    y=y>>4;
    printf("x=%d\t\tty=%u\n",x,y);
    printf("x: "BYTE_TO_BINARY_PATTERN"\ty:"BYTE_TO_BINARY_PATTERN"\n",
    BYTE_TO_BINARY(x) ,BYTE_TO_BINARY(y));
}
```

```
void printBits(size_t const size, void const * const ptr)
{
    // Assumes little endian
    unsigned char *b = (unsigned char*) ptr;
    unsigned char byte;
    int i, j;
    //prints MSB first: remark start with index 7
    for (i = size-1; i >= 0; i--) { //for each byte
        for (j = 7; j >= 0; j--) { //for each bit
            byte = (b[i] >> j) & 1; //extract bit
            printf("%u", byte);
        }
        puts("");
    }
}

int testPrint(){
    int i = 23;
    unsigned int ui = UINT_MAX;
    float f = 23.45f;
    long l=INT_MIN;
    printf("int i=23\t");printBits(sizeof(i), &i);
    printf("UINT_MAX\t");printBits(sizeof(ui), &ui);
    printf("float 23.45\t");printBits(sizeof(f), &f);
    printf("long l=INT_MIN\t");printBits(sizeof(l), &l);
    return 0;
}

int main(){
    int x=0b11111;
    printf("x er: %d=0x%x\n",x,x);
    printf("-----\n");
    shift();
    printf("-----\n");
    testPrint();
}
```

QUIZ Tal-repræsentation 1/4: bits bytes words

En bit kan repræsentere 2 mulige værdier, benævnt 0 og 1 (false og true).

Med 2 bits kan vi lave 4 forskellige kombinationer, og dermed 4 forskellige værdier:

bit 1	bit 0
0	0
0	1
1	0
1	1

En byte er en samling af 8 bits. Hvor mange forskellige værdier kan repræsenteres ved en byte?

Et ord (word) er en samling af et antal bits i en enhed, der repræsenterer et typeløst tal.

Hvor mange forskellige værdier kan repræsenteres af et ord, som er 2 bytes ($w=16$ bits) langt?

QUIZ Tal-repræsentation 2/4: bin-dec-hex

Binær til decimal: Hvilken decimal-værdi har det binære tal 01110110?

Answer:

Hex til decimal: Hvilken decimal (10-tals system) værdi har hexadecimal tallet 0xA5B7?

Answer:

Binær til hex: Hvilken hexa-decimal-værdi har det binære tal 01110110? 0x...

Answer:

Binær til hex: Hvilken hexa-decimal-værdi har det binære tal 110110? 0x...

Answer:

QUIZ Tal-repræsentation 3/4: bin-dec-hex

Angiv tallet 0xA5B7 på binær form (2-tals system).
(angiv svar uden whitespaces)

Answer:

Angiv tallet 0x9FF som et 16-bit binært ord.
(brug ingen whitespaces i svar)

Answer:

QUIZ Tal-repræsentation 4/4: hex-sum

Lad $a=0x28$.

Hvilken hexadecimal værdi har $a-0x8$? (angiv svar uden 0x prefix)

Hvilken hexadecimal værdi har $a+0x8$? (angiv svar uden 0x prefix)

Hvilken hexadecimal værdi har $a+0xf$? (angiv svar uden 0x prefix)

Quizz: Hukommelsen og adressering

Figuren nedenfor viser en model af hukommelsen med noget indhold (alle tal i hex):

Bytes

				48	65	6A	20	6D	65	64	20	6A	65	73	21	0	...	
Addr.	00000000	00000001	00000002	00000003	00000004	00000005	00000006	00000007	00000008	00000009	0000000A	...					ffffff	

Antag at hukommelsen er 2^{32} bytes stor.

- Hvad er den mindste adresse i hex? 0x
- Hvad er den højeste adresse i hex? 0x
- Hvad er den højeste adresse som decimal:

Antag at strengen "Hej med jer!" er kodet op i [ascii](#) og er gemt i hukommelsen med start i adresse 0x4. Da hver karakter fylder een byte giver det følgende sekvens af ascii værdier (i hex, inklusiv C's "0"-terminering af strenge): 48,65,6A, 20, 6d, 65, 64, 20, 6A, 65,73,21,0.

- Hvad er værdien af byten som er gemt i adresse 0xA? 0x....
- I hvilken adresse ligger karakteren "!" gemt? 0x...
- Hvad er decimal værdien af den byte som er gemt i adresse 0x8?
- Hvilken karakter er gemt i adresse 0x4+0xB?

QUIZZ

The 64-bit word 0x1A23F456BC23D57B is stored in memory starting at address 0x510.

The 64-bit word 0x1A23F456BC23D57B is stored in memory starting at address 0x510.

In a computer using LITTLE endian, what is the value of the byte stored in address 0x514 ? 0x..

Drag answer here

In a computer using BIG endian, what is the address containing the least significant byte 0x7B? 0x...

Drag answer here

In a computer using LITTLE endian, what is the address containing the most significant byte 0x1A? 0x...

Drag answer here

In a computer using BIG endian, what is the value of the byte stored in address 0x514 ? 0x..

Drag answer here

In a computer using LITTLE endian, what is the address containing the least significant byte 0x7B? 0x...

Drag answer here

In a computer using LITTLE endian, a 16-bit word starts at address 0x514 ? What hex-value is stored there? 0x..

Drag answer here

In a computer using BIG endians, what is the address containing the most significant byte 0xA1? 0x...

Drag answer here

510

517

56f4

f4

56

bc

f456

5a0

d5

QUIZZ

The 64-bit word 0x1A23F456BC23D57B is stored in memory starting at address 0x510.

In a computer using LITTLE endian, what is the value of the byte stored in address 0x514 ? 0x..

In a computer using BIG endian, what is the address containing the least significant byte 0x7B? 0x...

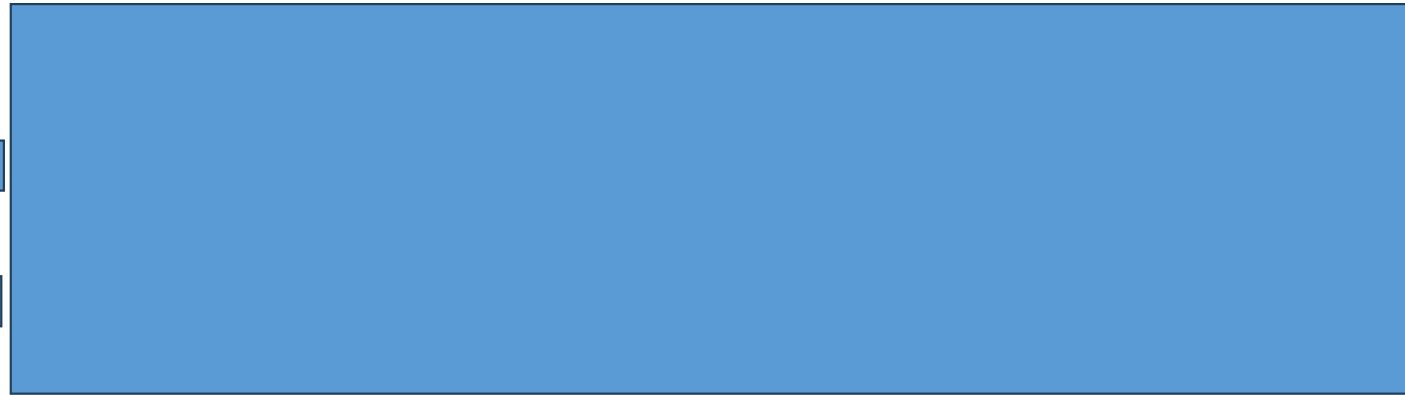
In a computer using LITTLE endian, what is the address containing the most significant byte 0x1A? 0x...

In a computer using BIG endian, what is the value of the byte stored in address 0x514 ? 0x..

In a computer using LITTLE endian, what is the address containing the least significant byte 0x7B? 0x...

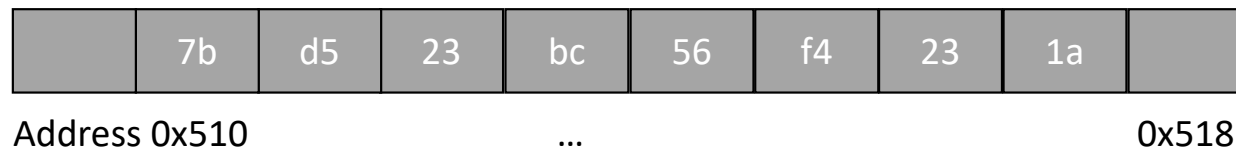
In a computer using LITTLE endian, a 16-bit word starts at address 0x514 ? What hex-value is stored there? 0x..

In a computer using BIG endians, what is the address containing the most significant byte 0xA1? 0x...



Further + Notation

- X86 er little End-ian
- ARM er bi-Endian, normalt konfigureret til little.



- Notation for læsning af hukommelsen ("Memory"):
 - $M_L[addr]$ → ordet af længe L bytes gemt i start adresse '**addr**'
 - $M_1[0x510]$ → 7b = 0x7b (little End fortolkning) = 0x7b (bigE)
 - $M_4[0x510]$ → 7bd523bc = 0xbc23d57d (little End fortolkning)
 - $M_8[0x510]$ → 7bd523bc56f4231a = 0x1a23f456bc23d57d (little End. fort.)